case3.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import org.springframework.web.bind.annotation.*;
import jakarta.persistence.*;

import java.util.List;
import java.util.Optional;

/**
 * Case Study 3: Book Management System
 *
 * Problem Statement:
 * A college library wants to maintain book records. The librarian should be able to
 * add books and retrieve all available book records from a database.
 *
 * Concepts: Spring Data JPA, JpaRepository, MySQL, CRUD
 *
 * Requirements:
 * 1. Store book ID, title, and author.
 * 2. Save book records in MySQL.
 * 3. Retrieve all books through a REST API.
 *
 * ------------------------------------------------------------
 * Required application.properties configuration for MySQL:
 * ------------------------------------------------------------
 * spring.datasource.url=jdbc:mysql://localhost:3306/library_db?createDatabaseIfNotExist=true
 * spring.datasource.username=root
 * spring.datasource.password=root
 * spring.jpa.hibernate.ddl-auto=update
 * spring.jpa.show-sql=true
 * ------------------------------------------------------------
 */

@SpringBootApplication
public class case3 {

    public static void main(String[] args) {
        SpringApplication.run(case3.class, args);
        System.out.println("Book Management System is running on http://localhost:8080");
    }
}

// ---------------- ENTITY CLASS (JPA) ----------------
@Entity
@Table(name = "books")
class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "title", nullable = false)
    private String title;

    @Column(name = "author", nullable = false)
    private String author;

    // Default constructor (required by JPA)
    public Book() {
    }

    // Parameterized constructor
    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public Book(Long id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
```

```
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    @Override
    public String toString() {
        return "Book [id=" + id + ", title=" + title + ", author=" + author + "]";
    }
}

// ---------------- REPOSITORY INTERFACE ----------------
@Repository
interface BookRepository extends JpaRepository<Book, Long> {
    // Custom finder method
    List<Book> findByAuthor(String author);
}

// ---------------- REST CONTROLLER (CRUD Operations) ----------------
@RestController
@RequestMapping("/books")
class BookController {

    // Dependency Injection of Repository
    @Autowired
    private BookRepository bookRepository;

    // 1. CREATE: Add a new book and save in MySQL (POST)
    // URL: POST http://localhost:8080/books
    /*
     * Sample JSON Body:
     * {
     *     "title": "Clean Code",
     *     "author": "Robert C. Martin"
     * }
     */
    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }

    // 2. READ ALL: Retrieve all books from MySQL database (GET)
    // URL: GET http://localhost:8080/books
    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    // 3. READ ONE: Retrieve a single book by ID (GET)
    // URL: GET http://localhost:8080/books/{id}
    @GetMapping("/{id}")
    public Book getBookById(@PathVariable Long id) {
        return bookRepository.findById(id).orElse(null);
    }

    // 4. UPDATE: Update book title/author (PUT)
    // URL: PUT http://localhost:8080/books/{id}
    @PutMapping("/{id}")
    public Book updateBook(@PathVariable Long id, @RequestBody Book updatedBook) {
        Optional<Book> optionalBook = bookRepository.findById(id);
        if (optionalBook.isPresent()) {
            Book existingBook = optionalBook.get();
            existingBook.setTitle(updatedBook.getTitle());
            existingBook.setAuthor(updatedBook.getAuthor());
            return bookRepository.save(existingBook);
        }
        return null;
    }

    // 5. DELETE: Delete a book record by ID (DELETE)
    // URL: DELETE http://localhost:8080/books/{id}
    @DeleteMapping("/{id}")
    public String deleteBook(@PathVariable Long id) {
        if (bookRepository.existsById(id)) {
            bookRepository.deleteById(id);
            return "Book with ID " + id + " has been deleted successfully.";
        }
        return "Book not found with ID " + id;
    }
}
```